

12 Retrieval-Augmented Generation and Text-to-SQL

This chapter differs somewhat from the rest of the book, as we're not presenting working techniques and tools that make programming easier for you, but rather developing a program that uses AI to achieve a result. Of course, there is still a connection: the code for the examples in this chapter was again created with AI support. So, this chapter again contains a lot of prompts as well as some comments on why we formulated the prompt in exactly the same way and why some of the previous attempts failed.

The problem for the following AI application is to analyze data from different sources using a large language model (LLM). We use PDF documents, web pages in HTML format, and an SQL database as basic data. In addition, two different techniques are used to realize these requirements: retrieval-augmented generation (RAG) and Text-to-SQL. The idea is to use these technologies to make a dormant treasure trove of data searchable.

In numerous projects, the keyword search for archived PDFs and other documents isn't sufficient, let alone the data in an SQL database. Asking questions in natural language and having them answered with the content of this data can really be a game changer in certain areas.

Retrieval-augmented generation (RAG) is one of the buzz words in the current AI hype. Because it's impossible to add your own information to a ready-made LLM with manageable effort, an additional small, custom language model, the *embedding* model, is placed in front of a large, general language model. When a query is made, the results of this model are redirected to the LLM, which then generates the response for the user.

The embedding model converts unstructured data, such as text or images, into multidimensional vectors. As with a geographical coordinate system

that describes points on the earth, these vectors provide information about the position of information in relation to each other. However, these vectors aren't limited to three dimensions as in a geographical coordinate system, but rather can have hundreds or thousands of dimensions.

Using Text-to-SQL, an LLM generates an SQL query from a query in natural language. For this to work, information on the database structure is transferred to the LLM along with the request. As you can imagine, the LLM only has a chance of formulating SQL queries that make sense if the table and column names have meaningful designations.

For the development of the AI application in the following example, we use the open-source library, *LlamaIndex* (www.llamaindex.ai). This library provides connections for the Python programming language as well as for TypeScript. We've chosen Python for this example.

12.1 RAG Quick Start

As an introduction to the topic, we want to show you how quickly you can get an executable AI application if the AI writes it itself. We want to ask questions on an HTML page about documents we store in a `pdfs` folder on the hard disk. These are annual reports and further information on a research project on insect censuses in Austria, which we'll discuss in more detail later in the chapter. Our prompt to the Claude 3.5 Sonnet LLM reads as follows:

Prompt: Generate a FastAPI backend for a LlamaIndex Q&A application and an HTML page to input questions. Data for LlamaIndex is in a folder `pdfs`. Don't use templating, but serve the HTML file from server root.

Unfortunately, the generated Python code isn't executable because the LlamaIndex library has changed somewhat since the end of the training data for Claude. This is an issue we've already encountered several times in this book. However, the changes are minimal (the `GPTSimpleVectorIndex` has been renamed `VectorStoreIndex`), and after changing three lines of code, the application runs. Here, we show you the entire backend script that starts the web server, answers questions using AI, and returns the answer in JavaScript Object Notation (JSON) format:

```

from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from llama_index.core import (
    VectorStoreIndex, SimpleDirectoryReader
)

app = FastAPI()

documents = SimpleDirectoryReader("pdfs").load_data()
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()

class Question(BaseModel):
    text: str

@app.post("/api/ask")
async def ask_question(question: Question):
    try:
        response = query_engine.query(question.text)
        return {"answer": str(response)}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

app.mount("/", StaticFiles(directory="static",
    html=True),
    name="static")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Most of the few lines of code relate to the web server, which can easily be set up using the FastAPI and Uvicorn libraries. HTML documents in the static folder are provided regularly via the web server. In the case of an HTTP POST request sent to the /api/ask URL, the content of the request gets mapped to the previously defined Question class. The text entry it contains is passed to the query_engine call.

This is where the magic happens: with just three lines of code, the LlamaIndex library creates the `query_engine`, which can search the content of PDF documents with queries in natural language. To do this, the `SimpleDirectoryReader` reads a folder with files, in our case, PDF documents. A `VectorStoreIndex` is then generated from these documents, from which the `query_engine` is derived.

We'll explain what happens in detail during these steps in the following sections. First, we want to try out our application. As Claude explains in the short explanation, we need to install the necessary Python libraries:

```
pip install fastapi uvicorn llama-index pydantic
```

The HTML frontend requires just a few lines of JavaScript code and no additional libraries. We don't need to make any changes to the file that we save as `index.html` in the `static` folder. The central part of the HTML file reads as follows:

```
<textarea id="question" placeholder="Enter your
question here">
</textarea>
<button onclick="askQuestion()">Ask</button>
<div id="answer"></div>

<script>

    async function askQuestion() {
        const question =
document.getElementById("question").value;
        const answerDiv =
document.getElementById("answer");
        answerDiv.innerHTML = "Loading...";
        try {
            const response = await fetch("/api/ask", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                },
                body: JSON.stringify({ text: question }),
            });
```

```

    });
    if (!response.ok) {
        throw new Error("Network response was not
ok");
    }
    const data = await response.json();
    answerDiv.innerHTML = data.answer;
} catch (error) {
    answerDiv.innerHTML = "Error: " + error.message;
}
}

</script>

```

The `askQuestion` function, which gets executed when the **Ask** button is clicked, sends the content of the text field with the question ID as a JSON string to the backend using an HTTP POST request. If the request is successfully answered, the content of the `data.answer` JSON structure gets inserted in the corresponding HTML-DIV area.

What Claude didn't mention in the description is that LlamaIndex swaps out the tough work to OpenAI. Thus, the application can only work if you have an OpenAI API key and available credit in your account. To give the application access to your account, you must set the API key in the `OPENAI_API_KEY` environment variable. In common Linux shells, you need to call the following command:

```
export OPENAI_API_KEY="sk-proj-xxxxxxx"
```

Now we can start our application using the following command:

```
python main.py
```

The result is quite convincing. After 10 minutes, the first web application is ready, answering questions in natural language about PDF documents on the solid-state drive (SSD).

Before you close the book (or switch off the e-reader) and get to work implementing this example, we want to point out a few shortcomings of this application:

- Each time you start the web server, the index gets regenerated. This, however, isn't necessary because you can also save a `VectorStoreIndex` locally.
- The default settings require you to have access to OpenAI and pay for it. However, LlamaIndex works with other cloud providers and also with local LLMs, and it's not at all bad at that.
- You can set very precisely which types of documents you want to index and how the indexing works in detail. There is great potential for optimization here.

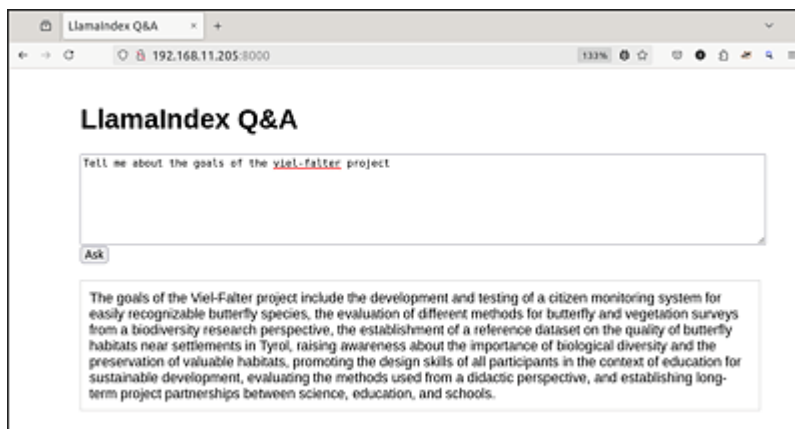


Figure 12.1 Our Quick Introduction to RAG

When reading the following sections, you'll get a deeper insight into how RAG and the LlamaIndex library work.